ASSIGNMENT REPORT

COM-601 Artificial Intelligence with Computer Vision

on

Retail Analytics - Identifying Products or Customers in Retail Environments

Submitted by

Aman Sharma
2023A1R042

Submitted to

Dr. Sakshi Popli

Department of Computer Science and Engineering

Model Institute of Engineering and Technology (Autonomous)

Jammu, India

Year **2026**

Assignment Description

Students are required to design and develop a basic Artificial Intelligence–based system capable of identifying and tracking objects using Computer Vision techniques. The proposed system should be able to detect objects from images or a webcam input and track their movement across consecutive frames. When an object such as a pen, bottle, book, mobile phone, or person is presented to the camera or appears in an image, the system should correctly identify the object and display a bounding box along with the object label. Students must design the system to detect a minimum of four and a maximum of six object categories. Object detection may be performed using images, photographs, or real-time webcam input. The implementation should use Computer Vision techniques and tools, such as OpenCV, along with pre-trained deep learning models or other appropriate frameworks. Students may choose one of the following application domains to guide the design of their system:

• Surveillance Systems – detecting and tracking people or suspicious objects in security footage

• Smart Home Systems – identifying household objects or human activities

• Robotics Applications – detecting objects for robotic interaction and navigation

• Retail Analytics – identifying products or customers in retail environments The selected application domain should influence the choice of objects, dataset, and system design.

Date of Submission: 27 Apr 2026

Certificate / Declaration

I, **Aman Sharma(2023A1R042)** , hereby declare that the work presented in this report, titled "Development of an Object Identification and Tracking System Using Computer Vision," is my original work carried out for the completion of the assignment in the course COM-601 Artificial Intelligence with Computer Vision. This report has been prepared and submitted to my teacher as part of the academic requirements of the course. All sources of information, documentation, datasets, research papers, and technical references used in this work have been properly acknowledged.

Signature of the Student Dated: 27-04-2026

Aman Sharma

2023A1R042

Acknowledgements

I would like to express my sincere gratitude to my faculty guide, Dr. Sakshi Popli, for her guidance, support, and academic direction during the completion of this assignment. Her instructions helped me approach the problem not only from the coding perspective but also from the viewpoint of system design, practical understanding, and technical reporting. I would also like to thank the creators and maintainers of OpenCV, Ultralytics, PyTorch, and the COCO dataset, whose open-source tools and documentation made this project possible. Finally, I am thankful to my department and institute for providing an environment where I could explore practical applications of Artificial Intelligence in real-world scenarios.

Aman Sharma

2023A1R042

Abstract

Retail environments generate continuous visual data in the form of customer movement, product handling, shelf interactions, and store activity. Monitoring this information manually is difficult, repetitive, and inefficient. As a result, computer vision has emerged as an important tool for modern retail analytics. By using object detection and tracking techniques, a visual system can identify customers and products, observe their movement, and help in operational analysis.

This report presents the design and implementation of a basic AI-based object identification and tracking system for the retail analytics domain. The developed system is capable of detecting seven selected object categories relevant to retail environments: `person`, `bottle`, `cup`, `cell phone`, `book`, `laptop`, and `banana`. The system accepts input from a webcam, an image, or a video and returns detections in the form of bounding boxes, labels, confidence scores, and tracking IDs when stream-based input is used.

The implementation is based on Python, OpenCV, Ultralytics YOLO, and PyTorch. A pre-trained YOLOv8 Nano model trained on the COCO dataset is used for detection and tracking. This report discusses the problem statement, motivation, dataset source, data characteristics, preprocessing steps, model selection, architecture, implementation details, observed results, limitations, and future scope. The work does not involve training a custom model from scratch; instead, it focuses on practical deployment and understanding of a complete computer vision pipeline.

The results show that the system performs successfully for assignment-level use and can detect the selected categories in common real-world conditions. It is fast enough for live demonstration and simple enough to understand from an academic perspective. Although the model has limitations such as generic class recognition and reduced performance in difficult conditions, the project demonstrates the practical usefulness of computer vision in retail analytics and provides a foundation for future enhancement.

Table of Contents

S. No.	Content	Page No.
1	Assignment Description	i
2	Certificate / Declaration	ii
3	Acknowledgement	iii
4	Abstract	iv
5	Introduction	1 – 2
6	Problem Statement	2 – 3
7	Objectives of the Project	3 – 5
8	Application Domain and Scope	5 – 6
9	Literature Review	6 – 9
10	Data Collection / Accumulation	9 – 10
11	Dataset Description	10 – 12
12	Data Shape, Type, and Feature Structure	12 – 13
13	Data Preprocessing	13 – 15
14	Model Selection and Justification	15 – 16
15	System Design and Architecture	16 – 18
16	Implementation Details	18 – 21
17	Results Obtained and Discussion	21 – 23
18	Image Results	23
19	Video Results	24
20	Limitations of the Model	24 – 26
21	Future Scope	26 – 27
22	Project & Source Code Link	27
23	Conclusion	27 – 28
24	References	28

LIST OF FIGURES

Figure Serial No.	Caption	Page No.
1	Workflow of YOLO-Based Object Detection System	17
2	Input image containing multiple selected objects	23
3	Output image showing detected objects with bounding boxes and labels	23
4	Input frames from video source	24
5	Output frame from video showing detection and tracking	24

LIST OF TABLES

Table Serial No.	Caption	Page No.
1	Comparative View of Models	9
2	Selected object categories	11

1. Introduction

Artificial Intelligence has become one of the most influential technologies of the present era. It is transforming the way systems process data, automate decisions, and interact with the real world. One of its major subfields is computer vision, which focuses on enabling machines to interpret images and video streams. Instead of depending on manual observation, computer vision systems allow computers to detect objects, understand scenes, and derive useful information from visual data.

Computer vision is used across many sectors such as healthcare, surveillance, transportation, robotics, agriculture, manufacturing, and retail. In retail, cameras are already present in many locations such as entrances, billing counters, shelves, storage rooms, and walking areas. These cameras are often used only for passive recording. However, with the help of Artificial Intelligence, the same video data can become a source of meaningful analytics. A smart system can estimate the number of customers, detect whether products are visible, monitor movement, and identify patterns that are useful for store management.

Retail analytics usually refers to the analysis of store operations, inventory, customer behavior, and product movement. Traditional retail analytics mainly depends on sales records, inventory data, and customer transaction history. While these sources are useful, they do not describe what is visually happening inside the store. Visual retail analytics fills this gap by analyzing customer presence, shelf interaction, and product visibility. It can help in better decision-making, store arrangement, loss prevention, and overall operational efficiency.

The main computer vision tasks relevant to this domain are object detection and object tracking. Object detection identifies an object category and locates it in the image through a bounding box. Object tracking extends this by attempting to preserve the identity of the object as it moves across multiple frames. In a retail context, this means a system can not only detect a customer or product in a frame but can also follow that object over time.

In this assignment, a basic prototype has been developed for the retail analytics domain. The system detects the classes `person`, `bottle`, `cup`, `cell phone`, `book`, `laptop`, and `banana` using a pre-trained YOLOv8 Nano model. These classes were selected because they are available in the COCO dataset and are suitable for demonstrating a retail analytics use case. The system works with webcam, image, and video input and is capable of displaying bounding boxes, labels, confidence values, and tracking IDs where applicable.

This project is important not only because it produces working code but because it brings together several concepts in one pipeline. It includes domain selection, data understanding, preprocessing, model comparison, implementation, visualization, and analysis of results. Therefore, it serves as a complete applied example of how computer vision techniques can be used in a real-world inspired scenario.

2.Problem Statement

Retail stores contain dynamic scenes where customers move continuously, products appear in different positions, and many types of visual interactions occur. Monitoring such environments manually is difficult and unreliable when long observation times are required. A human observer may miss important events, become fatigued, or be unable to monitor multiple locations consistently. Therefore, an intelligent automated system is needed to identify important objects and observe their movement.

The problem addressed in this project is to develop a basic Artificial Intelligence based system capable of identifying and tracking selected retail-related objects from visual input. The system should be able to process images, video files, and webcam streams, detect the chosen object categories, display bounding boxes around them, and assign labels. In the case of continuous input such as webcam or video, the system should also maintain tracking IDs across frames whenever possible.

More formally, the problem statement can be written as follows:

To design and implement a computer vision system for retail analytics that can detect selected object categories relevant to a retail environment and track them across frames, while presenting the output in an interpretable visual form.

This problem includes several challenges. First, the system must handle different types of input sources. Second, it must use a model that is fast enough to operate on live input without unacceptable delay. Third, the system must filter detections so that only the relevant object categories are shown. Fourth, the system must present results clearly to the user using bounding boxes and labels. Finally, the solution must remain simple enough to explain as an academic project and efficient enough to run on ordinary hardware.

The chosen solution addresses these challenges by combining OpenCV for image and video handling with a pre-trained YOLO object detector and built-in tracking support. The final system is not meant to be a commercial retail platform, but it serves as a practical example of AI-based object identification and tracking within a retail analytics context.

3. Objectives of the Project

The broad objective of this project is to build a working object identification and tracking system for retail analytics using computer vision. This overall objective can be divided into several focused goals that guided the design and implementation of the project.

The first objective is to study the role of computer vision in retail applications and understand how object detection and tracking can contribute to retail analytics. This is important because the technical design should be motivated by a realistic application domain rather than by code alone. The second objective is to compare different object detection models such as YOLO, Faster CNN, and SSD, and to choose a model that is suitable for an assignment-level real-time demonstration.

Another important objective is to select a set of object categories that are both relevant to the retail domain and available in a public dataset. For this reason, the categories `person`, `bottle`, `cup`, `cell phone`, `book`, `laptop`, and `banana` were selected. Together, these categories represent customers, handheld items, electronic items, and common product-like objects in a retail-like environment.

The project also aims to use a publicly available dataset source and a pre-trained model rather than training from scratch. This objective is practical because custom training requires significant computational resources, more time, and a larger annotation effort. Since the assignment allows the use of pre-trained models, the focus can remain on system design, integration, and demonstration.

Further objectives include implementing the complete system in Python, using OpenCV for media handling, using YOLO for detection and tracking, and ensuring that the system supports multiple input modes including webcam, image, and video. Another objective is to present results in a visually understandable way through bounding boxes, object labels, confidence values, and IDs. Finally, the project aims to analyze the strengths and limitations of the implemented solution and identify directions for future improvement.

From an academic perspective, these objectives are useful because they connect theory with practice. The project does not simply discuss computer vision models in abstract terms; it demonstrates how such models are selected, deployed, and interpreted in a real application context. This makes the assignment more meaningful and helps build an applied understanding of Artificial Intelligence. Each objective therefore contributes to the development of a complete mini-project rather than an isolated coding exercise.

Another important aspect of these objectives is that they are interdependent. The choice of classes affects the dataset relevance. The dataset relevance affects the suitability of the model. The model affects speed and output quality. The visualization stage affects the clarity of the final result.

Therefore, the project should not be viewed as a collection of separate tasks, but as a connected pipeline in which each design choice influences the next stage.

4. Application Domain and Scope

The application domain selected for this project is **Retail Analytics**. This domain focuses on understanding store activity, customer presence, and product-related observations using data-driven and vision-based techniques. Retail analytics systems are valuable because they help store managers improve operations, monitor movement, understand customer behavior, and make decisions related to inventory and layout.

In the present assignment, the scope of the project is intentionally limited to a basic but complete working prototype. The purpose is to demonstrate the core idea of identifying and tracking relevant objects, rather than to build a full-scale enterprise retail solution. The system is therefore designed to detect visible customers and a few selected object types that can represent products or customer-carried objects.

Within this scope, the system accepts three forms of input: webcam stream, image file, and video file. It performs object detection on each input and, in the case of video or webcam mode, also performs tracking so that detected objects can retain their identity over time. The final output is visual and interpretable, containing bounding boxes, class labels, and confidence scores.

It is important to clarify what is not included in the scope. The system does not recognize exact product brands, does not perform billing or inventory forecasting, does not support multi-camera synchronization, and does not include advanced analytics dashboards. It also does not perform customer re-identification or face recognition. These tasks require more advanced models, larger datasets, stronger hardware, and a much broader system design.

Even with these limitations, the project remains useful as a prototype because it demonstrates the essential perception layer of a retail analytics system. If a system can reliably detect and track relevant objects, more advanced analytics can later be built on top of that foundation.

The selection of retail analytics as the application domain is especially appropriate because it is easy to understand and easy to demonstrate. Almost everyone is familiar with the visual environment of stores, supermarkets, and malls. This makes it simpler to explain why object detection and tracking matter. A person entering the frame can represent a customer. A bottle or banana can represent a grocery product. A book can represent a product category in a bookstore. A laptop or cell phone can represent electronics or customer-carried items.

Because of this familiarity, the domain helps the technical system feel grounded in real-world usage rather than remaining purely theoretical.

At the same time, the scope limitation is important from an engineering perspective. Many student projects become unclear because they try to include too many features at once. In this report, the scope has been intentionally controlled so that the implemented system remains complete, explainable, and reliable.

This is often better than designing a much larger but partially implemented solution. In that sense, the project demonstrates a realistic principle of software engineering: a smaller working system is more valuable than a much larger system that cannot be verified properly.

5. Literature Review

Object detection has become one of the most studied and practically useful tasks in computer vision. The objective of object detection is not only to identify what object is present in an image, but also to determine where it is located. Compared with simple image classification, object

detection is more informative because it returns both semantic and spatial information. This makes it especially suitable for environments where multiple objects may appear simultaneously, as in retail stores.

Over time, many object detection algorithms have been developed. Broadly, they can be divided into two-stage detectors and one-stage detectors. Two-stage detectors first generate candidate regions and then classify them. One-stage detectors perform localization and classification in a single pass. In this project, three commonly discussed algorithms are considered: YOLO, Faster R-CNN, and SSD.

5.1 YOLO (You Only Look Once)

YOLO is one of the most influential one-stage detection algorithms. The main idea behind YOLO is that the image should be processed only once through the network, and the network should directly predict bounding boxes and class probabilities. This makes the system very fast compared with earlier approaches that separated region proposal generation and classification into different stages.

The working principle of YOLO can be understood in a stepwise way. First, the input image is passed into the neural network, which extracts useful visual features such as edges, shapes, patterns, and object-level structures. Next, the model predicts candidate bounding boxes and assigns confidence scores and class probabilities to them.

Since multiple overlapping predictions can occur for the same object, a post-processing step called Non-Maximum Suppression is applied to remove duplicates and keep only the strongest detection. The final output is a set of object labels with their coordinates and confidence values.

YOLO is especially important in practical applications because of its speed. In real-time applications such as webcam systems, autonomous systems, or live retail monitoring, speed is

often as important as raw accuracy. YOLO also provides an end-to-end architecture, making it easier to integrate into applications. However, it also has limitations. Small objects, heavily occluded objects, or cluttered scenes may reduce detection quality. Lightweight variants are faster but may sacrifice some accuracy.

5.2 Faster R-CNN

Faster R-CNN is a well-known two-stage object detector and remains one of the most important models in the history of deep learning based detection. In the first stage, it uses a Region Proposal Network to generate candidate object regions. In the second stage, these proposed regions are classified and their bounding boxes are refined. This approach is usually slower than YOLO but often produces strong accuracy and localization quality.

Faster R-CNN is effective in more complex settings because its proposal-based strategy can focus more carefully on object regions. It is often used in research and applications where speed is less critical than accuracy. However, because this assignment requires live or near real-time webcam operation on common hardware, Faster R-CNN is not the most convenient option for the present use case.

5.3 SSD (Single Shot Detector)

SSD is another one-stage detection model. It predicts objects at multiple scales by using feature maps of different resolutions. This helps it detect objects of various sizes. SSD provides a compromise between speed and accuracy, making it a useful model in many deployment settings. However, compared with modern YOLO implementations, SSD is often less attractive in assignment projects because the YOLO ecosystem is currently more convenient and widely used for practical demos.

5.4 Comparative View of Models

Model	Type	Speed	Accuracy	Suitability for This Project
YOLO	One-stage	Very High	High	Very suitable
Faster R-CNN	Two-stage	Lower	Very High	Less suitable
SSD	One-stage	High	Moderate to High	Suitable but secondary

From the literature and practical comparison, YOLO clearly emerges as the best fit for this project. The assignment requires real-time or near real-time performance, ease of integration, and good quality results on common object categories. The Ultralytics implementation of YOLOv8 satisfies these needs effectively.

6. Data Collection / Accumulation

In many machine learning projects, data collection involves manually gathering, labeling, and organizing training data. In this project, however, the system uses a pre-trained detection model. Therefore, data collection must be understood in two different ways. The first is the source of the model's learned knowledge, and the second is the set of practical inputs used to test and demonstrate the project.

The pre-trained model used in this work is trained on the COCO dataset. COCO is a large and publicly available computer vision dataset containing images of everyday scenes and objects. It includes many common categories such as people, bottles, cups, books, and mobile phones. Because all selected project classes are available in COCO, it becomes a suitable source for this retail analytics prototype.

For testing and demonstration, the system uses webcam input, image files, and video files collected by the user. These inputs are not used to retrain the model but are essential for validating that the system works correctly. The image files are useful for generating clean screenshots for the report.

The webcam is useful for real-time testing. The video files are useful for demonstrating tracking behavior over a sequence of frames.

This combination of public pre-trained knowledge and local demonstration input is appropriate for an assignment project. It allows the implementation of a practical AI pipeline without the time and resource demands of full model training.

Another reason why this approach is suitable is that it reflects a common real-world development workflow. In many practical AI applications, developers do not always begin by training a custom model. Instead, they first build a proof of concept using a pre-trained model, test whether the chosen object categories are meaningful, and study the behavior of the system under realistic conditions. If the prototype proves useful, custom data collection and fine-tuning may be introduced later. Therefore, even though this project does not include custom annotation and model training, its workflow still reflects a realistic and professional development path.

Data accumulation in the present project also includes collecting useful example scenes for evaluation. In retail-like environments, detection is affected by object placement, object scale, and background complexity. Therefore, testing the model using different images and videos is itself an important part of understanding how the system behaves. A single image is not enough to validate a detection system. By using multiple input formats, the project gains a more complete demonstration of its strengths and limitations.

7. Dataset Description

The COCO dataset, or Common Objects in Context dataset, is one of the most widely used datasets in modern computer vision research and practice. It contains a large number of images collected from real-world scenes where multiple objects appear in natural context. Unlike highly controlled datasets, COCO is valuable because objects appear under different lighting conditions, scales, backgrounds, and positions.

Each image in COCO may contain one or more objects, and each object is annotated with category labels and bounding boxes. In many cases, segmentation data is also available. Because the dataset represents common objects in realistic scenes, it is especially useful for general-purpose object detection models.

The selected object categories for this project are shown below.

Category	**Why It Was Selected**
person	Represents customers or staff in a store
bottle	Represents common packaged products
cup	Represents beverage-related or shelf objects
cell phone	Represents customer-carried item
book	Represents sellable item in some retail contexts
laptop	Represents an electronic item useful for retail display or customer-carried observation
banana	Represents a grocery-style product category

These classes were chosen because they are both available in the model and relevant to the chosen application domain. Even though COCO is not a retail-specific dataset, it supports enough real-world object categories to make it suitable for a basic retail analytics assignment.

One of the major strengths of COCO is contextual variety. Objects are not always shown in perfectly centered, clean, isolated form. Instead, they often appear in cluttered scenes, partially occluded, or surrounded by other objects. This makes the dataset challenging but also realistic. Because the present project aims to work on normal images and webcam scenes rather than laboratory-style input, a model trained on COCO is well suited for such a demonstration.

Another useful aspect of COCO is that it has become a standard reference dataset in computer vision. Using a COCO-trained detector makes the project easier to justify academically because

the dataset and the corresponding models are widely discussed in research papers, documentation, and practical tutorials. This gives the project both technical convenience and academic credibility.

8. Data Shape, Type, and Feature Structure

The teacher specifically asked for the shape of data, type of data, dataset description, and the number of features or columns. Since this project is built in the domain of computer vision, these concepts must be interpreted in the context of image data rather than spreadsheet-like data.

An image can be understood as a three-dimensional array of pixel values. Its shape is generally written as `(height, width, channels)`. For example, a 720p image may have the shape `(720, 1280, 3)`, where the last dimension represents the color channels. For video input, the system does not process the entire video as one matrix at once. Instead, it processes one frame at a time, and each frame has a similar image-like shape.

The type of data used in this project is primarily visual and therefore unstructured from the perspective of traditional tabular analytics. However, it also includes structured annotation information such as class IDs, class names, box coordinates, and confidence values. Therefore, the project combines unstructured image data with structured metadata and prediction outputs.

Unlike datasets such as those used in regression or classification problems with rows and columns, this project does not have a simple flat table of features like age, income, or salary. Still, the COCO annotation format contains fields such as `image_id`, `category_id`, `bbox`, `area`, `iscrowd`, and annotation `id`. Similarly, the output of the present system contains a set of meaningful attributes for each detected object: class label, confidence score, bounding box coordinates, and tracking ID when available.

Therefore, while the project is not based on a tabular dataset in the ordinary sense, its data structure can still be explained systematically through image dimensions, annotation fields, and detection output attributes.

This distinction is important because students are often taught data analysis using rows and columns, while computer vision uses a different representation. In image-based AI, useful features are not manually typed into a spreadsheet. Instead, the neural network learns features automatically from pixel patterns. Early layers learn simple features such as edges and textures, while deeper layers learn more complex semantic features such as object parts and object shapes. Therefore, the "features" of the project exist in a learned form inside the model rather than only as explicit columns visible to the user.

Understanding this difference is essential for explaining the project clearly in the final presentation. If someone asks how many features the dataset has, the most correct answer is that the project uses image tensors and structured annotations rather than a conventional row-column feature table. Still, the output can be described in terms of measurable attributes such as coordinates, confidence, and labels. This answer shows both conceptual clarity and domain-specific understanding.

9. Data Preprocessing

Data preprocessing is a necessary step in any AI pipeline because raw data is rarely in the exact form needed by the model. In this project, preprocessing does not involve building a large feature engineering pipeline, since the model is already trained. Instead, it focuses on preparing images and frames for inference and then cleaning the model output for display.

The first preprocessing step is image acquisition. The source may be a webcam index, an image path, or a video path. The program checks the source type and routes it to the appropriate handling function. If the source is an image, the system performs detection once. If the source is a webcam or video file, it enters a frame-processing loop.

The next step is reading the image or frames through OpenCV. OpenCV loads the input into memory as pixel arrays. Internally, the YOLO model standardizes the input for inference, which usually involves resizing. This standardization is important because deep learning models expect a consistent representation. Even though the resizing is handled internally by the framework, it is part of the effective preprocessing pipeline.

After the model produces detections, confidence filtering is applied. The system uses a confidence threshold so that extremely weak detections are not displayed. This improves clarity and reduces noise in the output. A second important step is semantic filtering. Because the model can detect many classes beyond the selected categories, the code explicitly keeps only the classes needed for the assignment and demonstration. This makes the system more domain-specific and easier to explain.

The project also includes optional mirror mode. When activated, the image is flipped horizontally before inference. This is mainly a usability enhancement for webcam demonstration because it makes the visual interaction feel more natural. Finally, the selected detections are converted into rectangles and text overlays so that the final output becomes easy to interpret visually. In the present version of the system, filtering is applied to retain the classes `person`, `bottle`, `cup`, `cell phone`, `book`, `laptop`, and `banana` only.

In summary, preprocessing in this project includes source resolution, image/frame reading, internal resizing, confidence filtering, class filtering, optional mirroring, and output visualization preparation.

Preprocessing is often underestimated because users focus mainly on the final model. In practice, however, preprocessing strongly affects the usability of the entire system. If the input is not handled properly, the model may still run, but the results may be harder to interpret or may not match the intended domain. By adding class filtering, the project aligns the general-purpose model

with the retail analytics theme. By adding mirror mode, it improves the presentation experience. By controlling confidence thresholding, it reduces weak or noisy results. These are all small engineering decisions, but together they significantly improve the quality of the final system.

It is also useful to distinguish between classic image preprocessing and AI-specific post-processing. Classic image preprocessing includes tasks such as reading images, flipping them, and formatting them for input. AI-specific post-processing includes thresholding predictions, extracting box coordinates, filtering classes, and rendering labels. The project uses both forms. This makes it a good demonstration of how traditional computer vision utilities and modern deep learning models work together in a complete application.

10. Model Selection and Justification

The choice of model is one of the most important decisions in this project because it affects both the accuracy and the practical usability of the final system. Since the assignment requires live webcam support and a working classroom demonstration, the selected model should be lightweight, efficient, and easy to integrate.

After comparing major options, the chosen model for this project is **YOLOv8 Nano**, represented by the file `yolov8n.pt`. This model belongs to the YOLO family and is specifically designed to be lightweight while still providing useful detection quality. The Ultralytics framework makes it especially convenient to load and use.

The decision was based on several criteria. First, the model should run on a regular student laptop without the need for high-end GPU hardware. Second, it should support live processing of webcam frames at a usable speed. Third, it should be easy to integrate with Python and OpenCV. Fourth, it should have pre-trained weights available on the selected categories. Fifth, it should be simple enough to explain in an academic report and presentation.

YOLOv8 Nano satisfies these requirements effectively. It offers fast inference, low resource consumption, and a clean API for detection and tracking. Faster R-CNN, while powerful, is heavier and slower, which makes it less convenient for real-time demonstration. SSD is also a reasonable option, but YOLOv8 provides a more practical balance for this project.

The use of a pre-trained model is also justified. Training a custom model from scratch would require a large labeled dataset, stronger hardware, and significant training time. Since the assignment allows pre-trained models, using `yolov8n.pt` is both acceptable and technically sensible.

There is also an educational reason for selecting a lightweight pre-trained model. In a course-based assignment, it is often more valuable to fully understand the stages of the system than to spend most of the effort on a long training procedure that may not even complete successfully on available hardware. By using a stable pre-trained model, more attention can be given to source handling, filtering, visualization, tracking, reporting, and analysis. This creates a more balanced and complete learning experience.

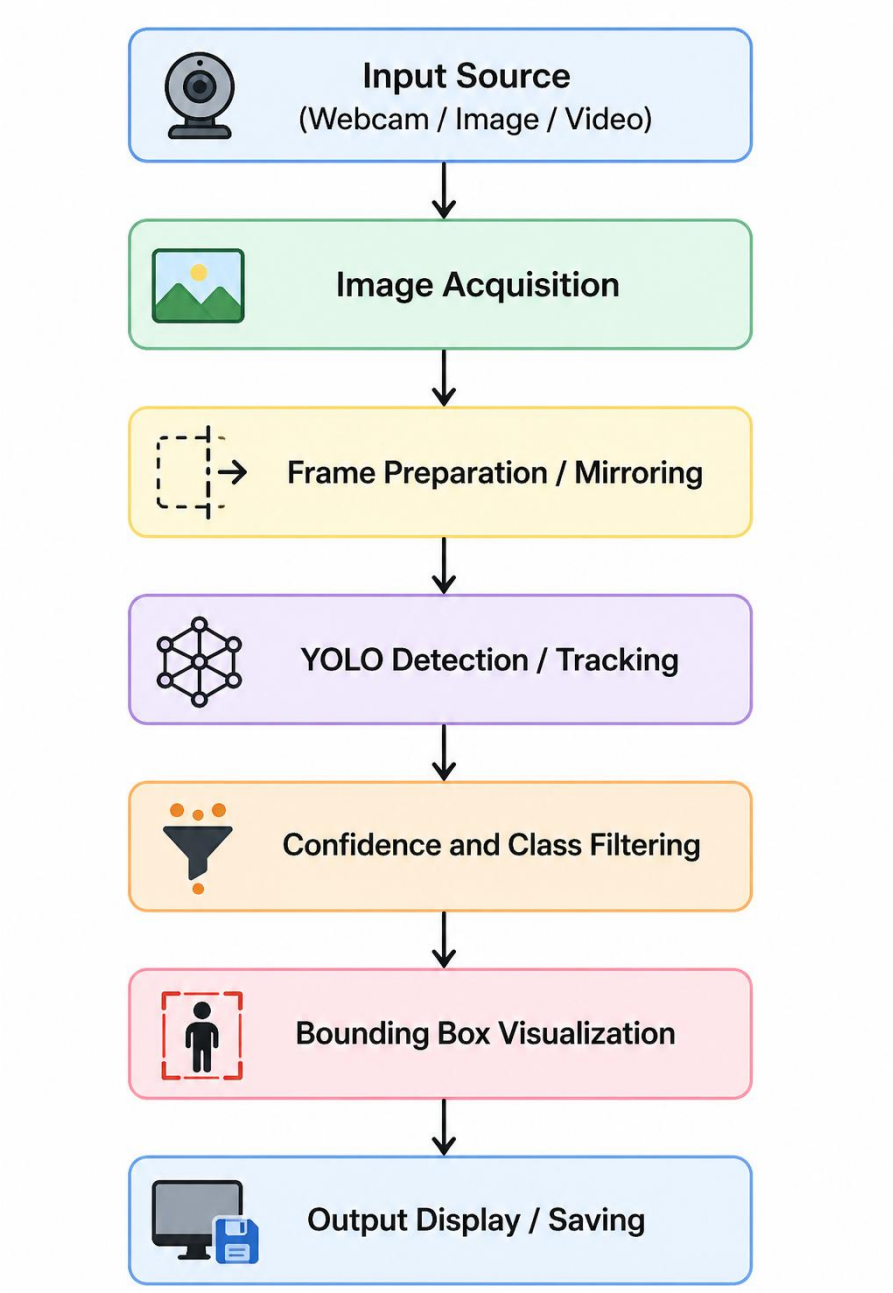
Furthermore, the chosen model is strong enough to produce meaningful detections for common classes without becoming too computationally expensive. This balance is especially important for live classroom or recorded demonstration. A model that is too slow may reduce the quality of the demonstration even if it is technically accurate. Therefore, the justification of the selected model is not based only on benchmark performance but also on practical usability and reliability.

11. System Design and Architecture

The developed system follows a straightforward computer vision pipeline. It begins by accepting an input source from the user, such as a webcam, image file, or video file. The input is then read into memory and prepared for inference. If mirror mode is enabled, the frame is horizontally

flipped. The prepared frame is then passed to the YOLO model for detection, or to the tracking pipeline in the case of continuous input. The output detections are filtered so that only the selected retail-related categories are retained. Finally, bounding boxes and textual labels are drawn, and the result is displayed or saved.

The complete system can be summarized by the following block diagram.



Workflow of YOLO-Based Object Detection System

The selected object classes also play an important role in the design. By restricting the system to `person`, `bottle`, `cup`, `cell phone`, `book`, `laptop`, and `banana`, the project becomes more clearly tied to the retail analytics domain. This design choice helps reduce unnecessary clutter in the output and makes the demonstration more focused. It also broadens the demonstration by covering customer presence, electronic items, and grocery-type products.

The architecture of the project is intentionally linear and easy to follow. This is beneficial in two ways. First, it makes the system easier to implement and debug. Second, it makes the project easier to explain during report writing and video presentation. Each stage has a clear role and produces output that becomes the input for the next stage. Such a pipeline structure is common in many practical computer vision applications and is therefore a good educational choice.

Another strength of the current architecture is modularity. Even though the codebase is small, the logical structure is separated into meaningful functions. This means that future features such as counting, logging, or advanced tracking could be added later without completely redesigning the application. In this sense, the architecture is simple but extensible.

12. Implementation Details

The implementation is written in Python and is primarily contained in the file `retail_analytics.py`. The overall code structure is modular enough to remain understandable while still being compact.

Python was chosen as the programming language because it is widely used in Artificial Intelligence and computer vision. Its syntax is simple, its ecosystem is mature, and it integrates well with both OpenCV and deep learning frameworks.

OpenCV is used for practical image and video handling. It reads image files, opens the webcam, captures frames from video files, draws rectangles, writes text labels, displays output windows,

saves processed outputs, and performs the mirror transformation. Without OpenCV, the deep learning model might still perform inference, but the system would not have a convenient user-facing way to manage visual input and output.

The Ultralytics library is used to load the YOLO model and run both prediction and tracking. This framework simplifies the use of YOLOv8 by providing a clean API. PyTorch is used underneath as the deep learning backend responsible for tensor operations and neural network inference.

The program begins by parsing command-line arguments through `parse_args()`. This function makes the script flexible by allowing the user to specify the source, confidence threshold, mirror mode, save option, and model path. The function `resolve_source()` checks whether the user has passed a webcam index such as `0` or a file path. The function `is_image_file()` determines whether the input is an image or a stream.

The function `draw_filtered_detections()` is central to the visual output. It reads the raw detections, extracts the class labels and confidence values, filters them according to the selected retail classes, retrieves tracking IDs when available, and draws bounding boxes with text. This function is what turns raw model output into a user-friendly result.

The function `run_on_image()` handles image mode. It loads the image, applies mirror mode if requested, runs prediction, annotates the detections, displays the image, and saves it if required. The function `run_on_stream()` handles webcam and video input by reading frames continuously, applying tracking, drawing detections, and updating the display until the stream ends or the user presses `q`.

The `main()` function coordinates the whole program. It parses arguments, loads the model, resolves the source, and routes execution to either image mode or stream mode. This structure keeps the project readable and suitable for explanation during viva or video presentation.

From a software engineering perspective, this implementation is intentionally small and practical. The code avoids unnecessary complexity and focuses on the core requirements of the assignment. This is useful because it reduces the chance of hidden bugs and makes the project easier to test. It also helps the student explain the code more confidently because each function has a clear and limited responsibility.

The use of command-line arguments is another useful design decision. Instead of hardcoding a webcam-only solution, the script accepts a flexible source argument. This makes the program reusable across different testing scenarios. It also improves the professionalism of the project because the same script can be demonstrated in multiple modes without modification.

The visualization logic is also worth noting. A model output by itself is only a set of numbers. The project converts those numbers into rectangles, labels, and confidence values that can be understood immediately by the viewer. This step is essential because the purpose of the assignment is not only to run inference but to present meaningful visual evidence of detection and tracking.

Typical commands used to run the project are shown below.

Webcam with mirror mode:

powershell

```
.\.venv\Scripts\python retail_analytics.py --source 0 –mirror
```

Image input:

powershell

```
.\.venv\Scripts\python retail_analytics.py --source ".\images\myphoto.jpg"
```

Video input:

powershell

```
.\.venv\Scripts\python retail_analytics.py --source ".\videos\myvideo.mp4"
```

13. Results Obtained and Discussion

The system was tested using webcam input, image input, and video input. Since this is an assignment prototype and not a benchmark study, the results are evaluated qualitatively rather than through large-scale statistical metrics. The main question is whether the system meets the assignment requirements and behaves correctly in practical use.

In image mode, the model successfully detected the selected classes when they were clearly visible in the scene. The output displayed a bounding box and label, which made it suitable for report screenshots. Image mode is especially useful because the same input can be tested repeatedly and clean visual evidence can be captured.

In webcam mode, the system processed frames continuously and demonstrated that real-time detection was possible on ordinary hardware. The mirror option improved the naturalness of the interaction, especially during live demonstration. Person detection was generally the most stable because people are very well represented in the COCO dataset.

In video mode, the project showed the additional benefit of tracking. The system attempted to assign persistent IDs to objects across frames, which made the output more informative than simple per-frame detection. This is important for retail analytics because movement over time often matters more than one isolated frame.

Several qualitative observations can be made from the results. First, the system performs best when the object is clearly visible, reasonably large, and not heavily occluded. Second, generic object categories such as bottle and book can be detected well when their appearance is clear and distinct.

Third, tracking quality is better in simpler scenes with moderate movement and fewer overlapping objects. Fourth, poor lighting and clutter reduce performance, which is consistent with the known behavior of vision models.

The results demonstrate that the selected design is appropriate for the assignment. The system successfully satisfies the requirements of detecting multiple categories, displaying boxes and labels, and performing detection from image and webcam input. The addition of tracking further strengthens the practical relevance of the project.

When preparing the final Word document, screenshots should be inserted in this section. Useful screenshots include a webcam-based person detection example, an image-based product detection example, a video tracking example, and a frame with multiple simultaneous detections.

The results also show that practical system quality is not determined only by model choice. Input quality matters greatly. When the object is sharp, well lit, and occupies a clear portion of the frame, detection performance is usually reliable. When the object is too small, partially blocked, or visually similar to nearby background patterns, performance becomes less stable. This reinforces an important lesson in computer vision: model accuracy depends not only on the network, but also on the visual conditions of the scene.

Another useful observation is that the chosen final object set helps make the demo more compelling. The presence of classes such as `banana` and `bottle` supports the grocery-style aspect of retail analytics, while `laptop`, `cell phone`, and `book` make the demo more varied and relatable. The `person` class anchors the customer side of the scenario. Because of this mix, the system is able to demonstrate product-like objects, customer presence, and carried items within one coherent visual theme. From a presentation point of view, the results are also strong because they are easy to explain. A viewer does not need technical background to understand that a labeled box around a banana or laptop indicates successful detection. This makes the system suitable for oral explanation, demo recording, and academic evaluation.

13.1 Image Results

Input image containing multiple selected objects

Output image showing detected objects with bounding boxes and labels

13.2 Video Results

Input frames from video source

Output frame from video showing detection and tracking

14. Limitations of the Model

Although the project works successfully for educational purposes, it has several limitations that should be stated clearly.

The first limitation is that the model is generic rather than retail-specific. It can detect a bottle, but it cannot distinguish one brand or SKU from another. This is because the model is trained on the COCO dataset, which contains broad object categories rather than detailed product catalogs.

The second limitation is sensitivity to scene quality. If the lighting is poor, the object is too small, the image is blurry, or the object is partially hidden, the quality of detection may decrease. This is

particularly relevant in real store environments where products can be placed densely or partially blocked.

The third limitation concerns tracking. While the system can assign IDs across frames, tracking is not perfect. If two similar objects overlap, cross paths, or move rapidly, the ID may change. This is a known challenge in online tracking systems.

Another limitation is that the project focuses only on perception and visualization. It does not compute advanced retail metrics such as customer counting, zone-based heatmaps, shelf gap detection, or product interaction statistics. Those would require an additional analytics layer on top of the present system.

Finally, the project is designed for academic demonstration rather than industrial deployment. Real deployment would require stronger validation, more robust testing, privacy considerations, and possibly custom-trained models.

Another limitation is that the system does not reason about higher-level events. For example, it can detect that a person and a bottle are visible in the frame, but it cannot conclude whether the person is purchasing, picking up, returning, or comparing that item. Such semantic understanding would require more advanced event recognition logic and possibly additional temporal analysis beyond simple object tracking.

There is also a limitation in domain specificity. Even though the selected classes are useful for demonstration, they still represent broad categories. Real retail systems often need to know exactly which product is being handled, what brand it belongs to, and whether it is placed in the correct location. The current model is not capable of that level of precision because it is not trained on store-specific categories.

15. Future Scope

The present project provides a strong foundation for future improvement. One of the most important future directions is custom training or fine-tuning on a retail-specific dataset. This would allow the system to recognize actual store products rather than only broad object categories.

Another important direction is the addition of advanced tracking methods such as Deep SORT or ByteTrack. These methods can improve ID consistency in crowded scenes and make the system more useful for customer movement analysis.

The system can also be extended to support advanced retail analytics tasks such as customer counting, queue detection, zone occupancy measurement, and shelf monitoring. By storing detection and tracking data over time, it would be possible to build dashboards that summarize customer activity and product visibility.

Further improvements could include multi-camera support, edge deployment on dedicated hardware, cloud-based reporting, and privacy-aware design. All of these would move the current prototype closer to a real-world retail monitoring solution.

Future scope can also include a stronger evaluation methodology. Instead of relying mainly on visual inspection, later versions of the project could include a small annotated test set and use formal metrics such as precision, recall, and mean average precision for the chosen classes. This would make the system more scientifically evaluated and strengthen the analytical depth of the work.

Another useful direction is user-interface improvement. At present, the project focuses on direct OpenCV windows and saved outputs. In future work, the system could be connected to a dashboard where detections are counted, stored, and reviewed. This would make the solution more aligned with actual retail analytics workflows, where managers prefer summaries and trends rather than raw bounding boxes alone.

16. Project and Source Code Link :-

https://drive.google.com/drive/folders/1XTizRZ-rduGCfVQP5fihrEt9r5gprQcp?usp=sharing

17. Conclusion

This report presented the design and implementation of a basic object identification and tracking system for the retail analytics domain. The project was motivated by the need to analyze retail scenes automatically rather than relying only on manual observation. The selected object categories were `person`, `bottle`, `cup`, `cell phone`, `book`, `laptop`, and `banana`, and the system was designed to work with webcam, image, and video input.

The implementation used Python, OpenCV, Ultralytics YOLO, and PyTorch. A pre-trained YOLOv8 Nano model was selected because it offers a practical balance between speed and accuracy and is suitable for live demonstration. The report explained the complete workflow, including problem definition, literature review, data source, preprocessing, model selection, system design, implementation, results, limitations, and future directions.

The final system met the core requirements of the assignment. It detects multiple object categories, draws bounding boxes, displays labels, and supports tracking for continuous input. Although the system has limitations due to its generic pre-trained nature and lightweight design, it successfully demonstrates the usefulness of computer vision in retail analytics and provides a strong base for further development.

Overall, the project can be considered successful from both a technical and academic point of view. Technically, it delivers a working detection and tracking pipeline that can be demonstrated live. Academically, it shows understanding of model comparison, data interpretation, preprocessing, implementation design, and result analysis. This balance between working software and conceptual explanation is what makes the project a strong example of applied Artificial Intelligence with Computer Vision.

18. References

1. Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. "You Only Look Once: Unified, Real-Time Object Detection." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016.

2. Ren, S., He, K., Girshick, R., and Sun, J. "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks." IEEE Transactions on Pattern Analysis and Machine Intelligence, 2017.

3. Liu, W., Anguelov, D., Erhan, D., et al. "SSD: Single Shot MultiBox Detector." European Conference on Computer Vision, 2016.

4. OpenCV Documentation. https://docs.opencv.org/

5. Ultralytics Documentation. https://docs.ultralytics.com/

6. PyTorch Documentation. https://pytorch.org/

7. COCO Dataset Official Website. https://cocodataset.org/